

Claude Code 自定义状态栏（Statusline）完全指南

什么是 /Statusline

```
GLM-5.1 | 17:51 | Effort:medium | Thinking | v2.1.126
C:\Users\Alan
[CTX] [██████████] 49% 51% | Size:200K | In:7.8M Out:46K
Usage: In:98K Out:114 Crt:0 Rd:0
▶▶ bypass permissions on (shift+tab to cycle)
```

💡 Claude Code 的 **Statusline**（状态栏）是终端底部的信息条，可自定义显示模型名称、时间、工作目录、Git 分支、上下文使用率、API 用量等关键信息。

Claude Code 默认底部只有一行简单提示。通过配置自定义 `statusLine`，你可以将终端变成一个信息丰富的开发仪表盘，实时掌握：

- 当前使用的**模型**和推理**努力程度**
- **工作目录**和 **Git 分支**
- **上下文窗口**使用率和剩余空间
- **API 令牌**用量（输入/输出/缓存）
- **速率限制**消耗情况

核心概念速查

📄 理解以下术语是配置 Statusline 的前提。

术语	含义	数据来源
context_window	模型上下文窗口，即单次对话能处理的最大 token 数	Claude Code 内部计算

used_percentage	已使用上下文的百分比	input_tokens / context_window_size
remaining_percentage	剩余上下文百分比	100% - used_percentage
current_usage	当前请求的 token 用量明细 (嵌套在 context_window 内)	Anthropic API 返回
input_tokens	当前请求输入的 token 数量	API 响应
output_tokens	当前请求生成的 token 数量	API 响应
cache_creation_input_toke ns	写入缓存的 token 数量 (prompt caching)	API 响应
cache_read_input_tokens	从缓存读取的 token 数量	API 响应
rate_limits	Claude.ai 的速率限制状态 (5 小时/7天窗口)	Claude Code 内部
total_input_tokens / total_output_tokens	本次会话累计的输入/输出 token 数	Claude Code 内部累计

JSON 数据结构全景

 Statusline 脚本通过 **stdin 接收 JSON 数据**。理解数据结构是编写脚本的基础。字段位置和层级关系是踩坑高发区。

Claude Code 每次刷新状态栏时，会向配置的脚本通过 stdin 发送一个 JSON 对象。以下是**完整的数据结构**（基于 v2.1.126）：

代码块

```
1  {
2    "session_id": "942903a3-502b-4519-85e6-4292c8a0e1e0",
3    "transcript_path": "C:\\Users\\Alan\\.claude\\projects\\...\\xxx.jsonl",
4    "cwd": "C:\\Users\\Alan\\project",
5    "model": {
6      "id": "MiniMax-M2.7",
7      "display_name": "MiniMax-M2.7"
8    },
9    "workspace": {
10     "current_dir": "C:\\Users\\Alan\\project",
11     "project_dir": "C:\\Users\\Alan\\project",
```

```

12     "added_dirs": []
13 },
14 "version": "2.1.126",
15 "output_style": { "name": "default" },
16 "cost": {
17     "total_cost_usd": 15.70,
18     "total_duration_ms": 1617136,
19     "total_api_duration_ms": 384725,
20     "total_lines_added": 24,
21     "total_lines_removed": 0
22 },
23 "context_window": {
24     "total_input_tokens": 3087048,
25     "total_output_tokens": 10760,
26     "context_window_size": 200000,
27     "current_usage": {
28         "input_tokens": 56227,
29         "output_tokens": 86,
30         "cache_creation_input_tokens": 0,
31         "cache_read_input_tokens": 0
32     },
33     "used_percentage": 28,
34     "remaining_percentage": 72
35 },
36 "exceeds_200k_tokens": false,
37 "fast_mode": false,
38 "effort": { "level": "medium" },
39 "thinking": { "enabled": true },
40 "agent": {
41     "name": "reviewer",
42     "type": "code-reviewer"
43 },
44 "vim": { "mode": "NORMAL" },
45 "session_name": "feature-branch",
46 "rate_limits": {
47     "five_hour": { "used_percentage": 15, "resets_at": "..." },
48     "seven_day": { "used_percentage": 8, "resets_at": "..." }
49 }
50 }
51

```

顶层字段

- `session_id` : 会话唯一 ID
- `cwd` : 当前工作目录

context_window 嵌套字段

- `total_input_tokens` : 会话累计输入
- `total_output_tokens` : 会话累计输出

- `model`：当前模型信息
- `workspace`：工作区信息
- `version`：Claude Code 版本
- `cost`：累计成本统计
- `context_window`：**核心字段**
- `effort`：推理努力程度
- `thinking`：是否启用思考模式
- `agent`：当前 Agent 信息
- `vim`：Vim 模式状态
- `session_name`：会话名称
- `rate_limits`：速率限制
- `context_window_size`：窗口上限（默认 200K）
- `used_percentage`：已使用百分比
- `remaining_percentage`：剩余百分比
- `current_usage`：**当前请求用量（重要！）**
- `input_tokens`：当前输入
- `output_tokens`：当前输出
- `cache_creation_input_tokens`：缓存写入
- `cache_read_input_tokens`：缓存读取

安装配置流程

✅ 三步完成 Statusline 配置：创建脚本 → 配置 settings.json → 运行 /statusline 验证

创建 PowerShell 脚本

配置 settings.json

运行 /statusline 验证

调整样式和字段

步骤 1：创建脚本文件

在 `~/ .claude/statusline-command.ps1` 创建 PowerShell 脚本：

代码块

```
1 [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
2
3 $stdin = [System.Console]::OpenStandardInput()
4 $reader = [System.IO.StreamReader]::new($stdin)
5 $raw = $reader.ReadToEnd()
6 $reader.Dispose()
7
8 if ([string]::IsNullOrEmpty($raw)) {
9     Write-Output "Claude Code"
10     exit 0
11 }
```

```
12
13 # 解析 JSON
14 $d = $null
15 try {
16     $d = $raw | ConvertFrom-Json -ErrorAction Stop
17 } catch {
18     $d = $null
19 }
20
21 # 颜色定义
22 <equation>r = "</equation>([char]27)[0m"
23 function C(<equation>c) { "</equation>([char]27)[$c" }
24 $CR = C("1;31m")    # 红色
25 $CG = C("1;32m")    # 绿色
26 $CY = C("1;33m")    # 黄色
27 $CC = C("1;36m")    # 青色
28 $CM = C("1;35m")    # 洋红
29 $CW = C("1;37m")    # 白色
30 $CD = C("2m")       # 灰色
31
32 # 数字格式化
33 function Fmt($n) {
34     if ($null -eq $n) { return "0" }
35     if (<equation>n -ge 1000000) { return "</equation>([math]::Round($n /
36 1000000, 1))M" }
37     elseif (<equation>n -ge 1000) { return "</equation>([math]::Round($n /
38 1000))K" }
39     else { return "$n" }
40 }
41
42 # 安全属性访问
43 function Get-Prop($obj, $prop) {
44     if ($null -eq $obj) { return $null }
45     if ($obj -is [hashtable]) { return $obj[$prop] }
46     return $obj.$prop
47 }
48
49 # ===== LINE 1: 模型/时间/会话 =====
50 $l1 = @()
51 if ($d) {
52     $modelName = Get-Prop $d "model"
53     if ($modelName) {
54         $mn = if ($modelName -is [hashtable]) { $modelName["display_name"] }
55         else { $modelName.display_name }
56         if ($mn) { $l1 += "$CC$mn$r" }
57     }
58 }
```

```

56 $l1 += "<equation>CY</equation>(Get-Date -Format 'HH:mm')$r"
57
58 $sessionName = Get-Prop $d "session_name"
59 if ($sessionName) { $l1 += "$CM[$sessionName]$r" }
60
61 $agent = Get-Prop $d "agent"
62 if ($agent) {
63     $an = if ($agent -is [hashtable]) { $agent["name"] } else { $agent.name }
64     $at = if ($agent -is [hashtable]) { $agent["type"] } else { $agent.type }
65     if ($at) { $at = ":$at" } else { $at = "" }
66     if ($an) { $l1 += "$CG Agent:$an$at$r" }
67 }
68
69 $effort = Get-Prop $d "effort"
70 if ($effort) {
71     $el = if ($effort -is [hashtable]) { $effort["level"] } else {
$effort.level }
72     if ($el) { <equation>l1 += "</equation>([char]27)[1;34m Effort:$el$r" }
73 }
74
75 $thinking = Get-Prop $d "thinking"
76 if ($thinking) {
77     $te = if ($thinking -is [hashtable]) { $thinking["enabled"] } else {
$thinking.enabled }
78     if ($te) { $l1 += "$CY Thinking$r" }
79 }
80
81 # ===== LINE 2: 工作目录 =====
82 $l2 = @()
83 $workspace = Get-Prop $d "workspace"
84 if ($workspace) {
85     $cwd = if ($workspace -is [hashtable]) { $workspace["current_dir"] } else
{ $workspace.current_dir }
86     if ($cwd) {
87         # 替换中文用户名为英文
88         $cnName = ([char]0x738B) + ([char]0x4EAE)
89         $dispCwd = $cwd.Replace($cnName, 'Alan')
90         $l2 += "$CW$dispCwd$r"
91     }
92     if ($cwd) {
93         $branch = git -c core.quotepath=false -C "$cwd" rev-parse --abbrev-ref
HEAD 2>$null
94         if ($branch) { $l2 += "$CG($branch)$r" }
95     }
96 }
97
98 # ===== LINE 3: 上下文进度条 =====

```

```

99  $l3 = @()
100  $contextWindow = Get-Prop $d "context_window"
101  if ($contextWindow) {
102      $used = Get-Prop $contextWindow "used_percentage"
103      $rem = Get-Prop $contextWindow "remaining_percentage"
104      $size = Get-Prop $contextWindow "context_window_size"
105      $tIn = Get-Prop $contextWindow "total_input_tokens"
106      $tOut = Get-Prop $contextWindow "total_output_tokens"
107
108      if ($null -eq $used -and $null -ne $rem) { $used = 100 - $rem }
109      if ($null -eq $rem -and $null -ne $used) { $rem = 100 - $used }
110      if ($null -eq $used) { $used = 0 }
111      if ($null -eq $rem) { $rem = 0 }
112      if ($used -eq 0 -and $rem -eq 0) { $rem = 100 }
113      $used = [math]::Round($used)
114      $rem = [math]::Round($rem)
115
116      # Unicode 方块进度条
117      $barW = 15
118      $uChars = [math]::Floor($used / 100 * $barW)
119      if ($uChars -lt 0) { $uChars = 0 }
120      if ($uChars -gt $barW) { $uChars = $barW }
121      $rChars = $barW - $uChars
122
123      $filledBlock = [string][char]0x2588
124      $emptyBlock = [string][char]0x2591
125      $filled = "$CG" + ($filledBlock * $uChars)
126      $empty = "$CD" + ($emptyBlock * $rChars)
127      $bar = "$filled$empty$r"
128
129      $l3 += "$CW[CTX]$r $bar <equation>CG</equation>{used}%$r
<equation>CD</equation>{rem}%$r"
130      if ($size) { $l3 += "<equation>CC Size:</equation>(Fmt $size)$r" }
131      if ($tIn -or $tOut) {
132          $l3 += "<equation>CY In:</equation>(Fmt <equation>tIn) Out:</equation>
(Fmt $tOut)$r"
133      }
134  }
135
136  # Rate limits
137  $rateLimits = Get-Prop $d "rate_limits"
138  if ($rateLimits) {
139      $rp = @()
140      $fiveHr = if ($rateLimits -is [hashtable]) { $rateLimits["five_hour"] }
else { $rateLimits.five_hour }
141      $sevenDay = if ($rateLimits -is [hashtable]) { $rateLimits["seven_day"] }
else { $rateLimits.seven_day }

```

```

142
143     if ($fiveHr) {
144         $v = if ($fiveHr -is [hashtable]) { $fiveHr["used_percentage"] } else
{ $fiveHr.used_percentage }
145         if ($v) {
146             $v = [math]::Round($v)
147             $c = if ($v -gt 80) { $CR } elseif ($v -gt 50) { $CY } else { $CG }
148             $rp += "<equation>c 5h:</equation>{v}%r"
149         }
150     }
151     if ($sevenDay) {
152         $v = if ($sevenDay -is [hashtable]) { $sevenDay["used_percentage"] }
else { $sevenDay.used_percentage }
153         if ($v) {
154             $v = [math]::Round($v)
155             $c = if ($v -gt 80) { $CR } elseif ($v -gt 50) { $CY } else { $CG }
156             $rp += "<equation>c 7d:</equation>{v}%r"
157         }
158     }
159     if ($rp.Count -gt 0) {
160         $l3 += "$CW RL:$r $($rp -join ' ')"
161     }
162 }
163
164 # ===== LINE 4: 当前请求用量 =====
165 $l4 = @()
166 $contextWindow = Get-Prop $d "context_window"
167 if ($contextWindow) {
168     $currentUsage = if ($contextWindow -is [hashtable]) {
$contextWindow["current_usage"] } else { $contextWindow.current_usage }
169     if ($currentUsage) {
170         $inTok = Get-Prop $currentUsage "input_tokens"
171         $outTok = Get-Prop $currentUsage "output_tokens"
172         $cacheCreate = Get-Prop $currentUsage "cache_creation_input_tokens"
173         $cacheRead = Get-Prop $currentUsage "cache_read_input_tokens"
174
175         if ($currentUsage) {
176             <equation>l4 += "</equation>{CW}Usage:$r"
177             <equation>l4 += "</equation>{CY}In:$(Fmt $inTok)$r"
178             <equation>l4 += "</equation>{CG}Out:$(Fmt $outTok)$r"
179             <equation>l4 += "</equation>{CC}Crt:$(Fmt $cacheCreate)$r"
180             <equation>l4 += "</equation>{CM}Rd:$(Fmt $cacheRead)$r"
181         }
182     }
183 }
184
185 # ===== OUTPUT =====

```



```
186 [System.Console]::OutputEncoding = [System.Text.Encoding]::UTF8
187 $out = @()
188 if ($l1.Count -gt 0) { $out += ($l1 -join " | ") }
189 if ($l2.Count -gt 0) { $out += ($l2 -join " ") }
190 if ($l3.Count -gt 0) { $out += ($l3 -join " | ") }
191 if ($l4.Count -gt 0) { $out += ($l4 -join " ") }
192
193 if ($out.Count -gt 0) {
194     $final = $out -join "`n"
195     Write-Output $final
196 } else {
197     Write-Output "Claude Code"
198 }
199
```

步骤 2：配置 settings.json

编辑 `~/.claude/settings.json`，添加 `statusLine` 配置：

代码块

```
1 {
2   "statusLine": {
3     "type": "command",
4     "command": "powershell -NoProfile -ExecutionPolicy Bypass -File
  \"C:\\\\Users\\\\Alan\\\\.claude\\\\statusline-command.ps1\"",
5     "refreshInterval": 5
6   }
7 }
8
```

配置项	说明
type	固定为 <code>command</code>
command	调用脚本的完整命令
refreshInterval	刷新闻隔（秒），建议 5

步骤 3：验证

在 Claude Code 中运行：

代码块

```
1 /statusline
2
```

如果配置正确，底部状态栏会立即更新。

 **要点回顾：** Statusline 通过 stdin 接收 JSON，脚本解析后输出带 ANSI 颜色的多行文本。配置在 `settings.json` 中，刷新间隔建议 5 秒。

踩坑点全记录

 以下是我们配置过程中遇到的所有问题及解决方案。建议按顺序排查。

坑 1：System.IO.Console 类型错误

问题现象

脚本运行时报错：

代码块

```
1 无法找到类型 [System.IO.Console]
2
```

影响范围

- PowerShell 5.1（Windows 默认版本）
- PowerShell 7+ 不受影响

预防措施

始终使用 `[System.Console]` 而非 `[System.IO.Console]`。

原因

PowerShell 5.1 中 `System.IO.Console` 命名空间不存在，应该是 `System.Console`。

解决方案

代码块

```
1  # 错误
2  [System.IO.Console]::OutputEncoding = [System.Text.Encoding]::UTF8
3
4  # 正确
5  [System.Console]::OutputEncoding = [System.Text.Encoding]::UTF8
6
```

坑 2: current_usage 字段位置搞错

问题现象

第四行 Usage: 始终空白，没有任何数据显示。

原因

current_usage 不是顶层字段，而是嵌套在 context_window 内部的。直接 \$d.current_usage 会返回 \$null。

正确访问方式

代码块

```
1 $contextWindow = Get-Prop $d
  "context_window"
2 $currentUsage = if
  ($contextWindow -is [hashtable])
  {
3   $contextWindow["current_usage"]
4 } else {
5   $contextWindow.current_usage
6 }
7
```

调试方法

在脚本开头写入 debug 文件查看实际 JSON 结构：

代码块

```
1 $raw | Out-File
  "$env:TEMP\statusline_debug.json"
  -Encoding UTF8
2
```

关键发现

通过 debug 文件确认字段层级后再编写访问逻辑。

坑 3: 中文路径显示乱码

问题现象

工作目录中的中文用户名显示为 ?? 或乱码方块。

原因

终端编码与脚本输出编码不匹配。脚本需要显式设置 UTF-8 输出编码。

解决方案

代码块

```
1 # 脚本开头和结尾都要设置
```

进阶处理

如果不想显示中文用户名，可以替换为英文名：

代码块

```
1 $cnName = ([char]0x738B) +
  ([char]0x4EAE) # "xxx" 的
  Unicode
2 $dispCwd = $cwd.Replace($cnName,
  'Alan')
3
```

```
2 [Console]::OutputEncoding =  
  [System.Text.Encoding]::UTF8  
3 # ... 脚本逻辑 ...  
4 [System.Console]::OutputEncoding  
  = [System.Text.Encoding]::UTF8  
5
```

注意：直接用 `.Replace('xxx', 'Alan')` 可能因文件编码问题失效，建议用 Unicode 码点。

坑 4：缓存指标始终为 0

问题现象

`cache_creation_input_tokens` 和 `cache_read_input_tokens` 始终显示为 0。

原因

这些指标来自 Anthropic API 的 **prompt caching** 机制。使用第三方模型（MiniMax、DeepSeek、Kimi 等）或非标准 API 端点时，该机制可能未启用。

判断标准

- 原生 Claude 模型（Sonnet/Opus/Haiku）：可能有非零值
- 第三方模型：通常始终为 0

不影响使用

缓存指标为 0 不影响 Statusline 其他功能，只是缺少缓存效率信息。

替代方案

关注 `total_input_tokens` 和 `total_output_tokens` 来监控总体用量。

坑 5：颜色代码与 PowerShell 变量解析冲突

问题现象

字段名前面出现多余空格，或颜色不生效。

原因

PowerShell 字符串插值时，`$CY In:` 中的 `$C` 可能被误认为变量名开头。

解决方案

使用 `${VAR}` 语法明确变量边界：

代码块

其他注意事项

- ANSI 转义序列使用 `[char]27` 而非 `\e`
- 重置颜色用 `$r = "$([char]27)[0m"`
- 每个字段后都要加 `$r` 重置，避免颜色泄漏

```

1  # 错误：可能有前导空格
2  $l4 += "<equation>CY In:
    </equation>(Fmt $inTok)$r"
3
4  # 正确：使用花括号明确边界
5  <equation>l4 += "</equation>
    {CY}In:$(Fmt $inTok)$r"
6

```

坑 6: ConvertFrom-Json 返回 PSCustomObject 而非 hashtable

问题现象

访问嵌套属性时有时成功有时失败，行为不一致。

原因

PowerShell 5.1 中 `ConvertFrom-Json` 默认返回 `PSCustomObject`，而脚本中混用了 `.property` 和 `[$key]` 两种访问方式。

解决方案

编写统一的 `Get-Prop` 辅助函数：

代码块

```

1  function Get-Prop($obj, $prop) {
2      if ($null -eq $obj) { return
    $null }
3      if ($obj -is [hashtable]) {
    return $obj[$prop] }
4      return $obj.$prop
5  }
6

```

使用示例

代码块

```

1  $modelName = Get-Prop $d "model"
2  $mn = Get-Prop $modelName
    "display_name"
3

```

兼容性

此函数同时兼容 `PSCustomObject` 和 `hashtable` 类型。

✓ 踩坑速查

配置 Statusline 时，按顺序排查这 6 个常见问题，可解决 90% 的故障。

问题	关键词	解决方案

类型错误	System.IO.Console	改用 System.Console
字段空白	current_usage	从 context_window 内读取
中文乱码	UTF-8	脚本头尾设置 OutputEncoding
缓存为 0	cache_read	第三方模型不支持，属正常
前导空格	颜色变量	用 \${VAR} 语法
属性访问	ConvertFrom-Json	用 Get-Prop 统一封装

准确性问题



Statusline 显示的数据准确性取决于模型类型和 API 端点。以下问题需要了解。

上下文窗口大小固定为 200K

模型	实际上下文窗口	Statusline 显示	准确性
Claude Sonnet 4.6	200K	200K	准确
Claude Opus 4.7	200K	200K	准确
DeepSeek-V3.2	128K 或 256K	200K	不准确
Kimi-K2.6	256K 或 1M	200K	不准确
MiniMax-M2.7	模型相关	200K	不准确

结论： `context_window_size: 200000` 是 Claude Code 的**默认值**，第三方模型的实际限制可能不同。显示的 `used_percentage` 仅供参考。

百分比计算方式

代码块

```
1 used_percentage = current_usage.input_tokens / context_window_size * 100
2
```

注意：

- 只计算了 input_tokens，不包含 output_tokens
- 不包含 system prompt 和工具调用的 token
- 实际占用可能略高于显示值

速率限制仅限 Claude.ai

`rate_limits` 字段（5小时/7天窗口）只在以下场景有效：

- 使用 Claude.ai 官方 API
- 使用 anthropic 订阅账户

使用第三方 API 端点（如火山引擎 Ark）时，该字段可能不存在或始终为 0。

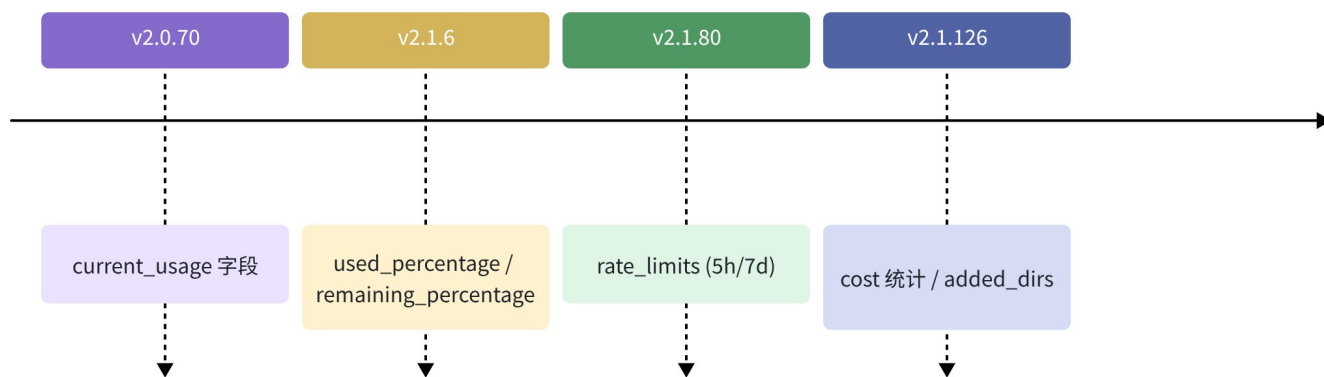
版本变更历史



Statusline 相关功能随 Claude Code 版本演进。以下是关键变更节点。

版本	日期	变更内容
v2.0.70	早期	新增 <code>current_usage</code> 字段，支持准确的上下文窗口百分比计算
v2.1.6	早期	新增 <code>context_window.used_percentage</code> 和 <code>context_window.remaining_percentage</code> ，简化进度显示
v2.1.80	早期	新增 <code>rate_limits</code> 字段（5小时/7天窗口），支持 Claude.ai 速率限制显示
v2.1.126	2026/05	当前版本，支持所有上述字段，含 <code>cost</code> 统计和 <code>added_dirs</code>

Statusline 功能演进



总结与学习路径

✅ Statusline 是 Claude Code 的隐藏利器，花 10 分钟配置，长期提升开发效率。

本文核心要点

1. **数据结构**: `current_usage` 嵌套在 `context_window` 内，不是顶层字段
2. **编码问题**: PowerShell 脚本必须设置 UTF-8 输出编码
3. **模型差异**: 第三方模型的缓存指标和上下文窗口大小可能不准确
4. **调试方法**: 用 `$raw | Out-File` 写入 debug 文件查看实际 JSON

下一步行动

- ☐ 复制本文脚本到 `~/.claude/statusline-command.ps1`
- ☐ 配置 `settings.json` 中的 `statusLine` 字段
- ☐ 运行 `/statusline` 验证效果
- ☐ 根据个人需求调整显示字段和颜色
- ☐ 关注 Claude Code 更新日志，及时获取新字段

参考链接

- [Claude Code 官方文档](#)
- [Claude Code GitHub](#)



文档版本: v1.0 | 基于 Claude Code: v2.1.126 | 最后更新: 2026/05/03